Software Requirements Specification: Project L.O.V.E. — The Experience Module

1. Executive Summary and Theoretical Foundation

1.1 Project Overview and Paradigm Shift

The Listener-Observer-Versor-Experience (L.O.V.E.) Project represents a foundational paradigm shift in the domain of Affective Computing. Historically, digital emotion recognition has been constrained by two-dimensional models—primarily the Valence-Arousal circumplex—which reduce the complexity of the human experience to static Cartesian coordinates. These models fail to capture the relational depth of the human condition. Project L.O.V.E. supplants this archaic framework with a system that maps human emotion as dynamic orientations within a three-dimensional, non-commutative algebraic space. By synthesizing the qualitative rigor of Dr. Brené Brown's *Atlas of the Heart* with the mathematical precision of quaternions, the system quantifies the "work" required to shift between emotional states.[1]
This document serves as the exhaustive Software Requirements Specification (SRS) for the **Experience Module**. The Experience Module constitutes the user-facing frontend of the L.O.V.E. Stack. It is not merely a passive dashboard but an immersive, bio-feedback instrument. Its primary directive is to visualize the user's emotional trajectory in real-time, rendering the "Soul Sphere"—a dynamic, reactive 3D object whose geometry, texture, chromatic properties, and motion are driven by the user's Valence, Arousal, and Connection (VAC) metrics.

1.2 The L.O.V.E. Stack Architecture

The system architecture operates as a reactive microservices loop, designed to mimic the neurological processing of emotional stimuli. The Experience module functions as the efferent nerve ending of this system:

1. **The Listener (Ingestion):** Ingests multi-modal input (voice/text) and normalizes it using Large Language Models (LLMs) and speech-to-text pipelines.
2. **The Observer (Context):** Manages state history and retrieves contextual embeddings from a vector database (pgvector).
3. **The Versor (Processing):** The mathematical engine that performs quaternion calculations, determining the shortest path rotation (SLERP) and angular velocity between states.
4. **The Experience (Visualization):** The subject of this SRS. It receives the state vectors and renders the biometric visualization while generating haptic feedback patterns.[1]

1.3 The VAC Model Integration

The Experience module is responsible for translating the abstract VAC metrics into tangible sensory outputs. The traditional "Dominance" axis of the VAD model has been replaced by "Connection" (z-axis), creating a new dimensional space crucial for modeling relational alignment.

Axis	Metric	Definition	Visual Mapping (Experience Module)
X	**Valence**	Intrinsic attractiveness (positive) or aversiveness (negative).	**Chromatic Interpolation:** Shifts color palette between crimson/grey (negative) and cyan/teal (positive).
Y	**Arousal**	Physiological stimulation and energy level.	**Vertex Displacement:** Controls the "roughness" or noise amplitude of the sphere's surface geometry.
Z	**Connection**	Degree of authentic alignment with self or others.	**Transmission/Opacity:** Controls the alpha channel and Fresnel glow intensity (opaque vs. radiant).

This mapping allows the system to visually distinguish complex states. For instance, "Pity" and "Compassion" may share Valence coordinates, but the Experience module renders them distinctively based on their opposing Connection vectors—Pity appears opaque and distant, while Compassion appears luminous and translucent.[1]

---

2. Technical Environment and Strategic Dependencies

2.1 The React Native Ecosystem Strategy

The selection of the underlying framework is the single most critical architectural decision for the Experience module. The requirement for high-fidelity 3D rendering on mobile devices necessitates a hybrid approach that leverages the declarative nature of React for UI logic while dropping down to native OpenGL/WebGL contexts for performance.

2.1.1 The "New Architecture" Compatibility Crisis

Recent evolutions in the React Native ecosystem, specifically the introduction of the "New Architecture" (Fabric renderer and TurboModules), have introduced significant instability in the WebGL ecosystem. Research indicates a critical conflict between the React Native New Architecture (enabled by default in version 0.76+) and the expo-gl library, which underpins React Three Fiber (R3F) on mobile.[2]

Developers attempting to run R3F with Fabric enabled frequently encounter the ExponentGLObjectManager error, resulting in a blank canvas or immediate application crash.[4] Furthermore, the asynchronous nature of the legacy bridge versus the synchronous execution of JSI (JavaScript Interface) in the New Architecture creates race conditions for frame loops that have not yet been fully resolved in the expo-gl C++ layer.[5]

Architectural Directive 2.1:

To ensure production-grade stability for the initial release of Project L.O.V.E., the Experience application must explicitly opt-out of the New Architecture. While the New Architecture offers performance benefits for 2D UI layouts, it is currently detrimental to 3D context creation.

- **Configuration:** The project must be initialized with the flag newArchEnabled=false in android/gradle.properties and via RCT_NEW_ARCH_ENABLED=0 in the iOS Podfile generation.[5]
- **Future Proofing:** The team shall monitor the expo-gl roadmap for C++ TurboModule support, aiming for migration in Phase 2 (post-MVP).

2.1.2 Dependency Versioning and Matrix

The 3D ecosystem in React Native is fragile regarding peer dependencies. A strict version matrix is required to prevent "dependency hell," particularly regarding the interaction between React 18, React 19, and the R3F reconciler.

Component	Package	Version Constraint	Technical Justification
Core Framework	react-native	0.76.x	Latest stable release, but must run in legacy bridge mode.[3]
Runtime	expo	~52.0.0	Provides the necessary expo-gl context bindings and asset management.[6]
3D Engine	@react-three/fiber	^8.17.x	**Crucial:** R3F v9 is in alpha and targets React 19. R3F v8 is stable for React 18.2.0.[7]
React	react	18.2.0	React 19 is not yet fully supported by the stable R3F ecosystem.[2]

Shaders	three-stdlib	Latest	Provides shader chunks and noise algorithms.
Haptics	react-native-haptics	Latest	**Critical:** Benchmarks show it is ~2.13x faster than expo-haptics for "Heavy" impacts, essential for syncing vibration to visual rotation.[8]
State	zustand	^4.5.x	Chosen for its ability to update state outside the React render cycle (transient updates), preventing frame drops during 60fps animation.

2.2 Hardware and Performance Constraints

The Experience module drives heavy GPU operations (custom vertex displacement shaders) concurrent with CPU-intensive tasks (physics calculations for haptics).

- **iOS Targets:** iPhone 11 (A13 Bionic) is the minimum baseline for full-resolution shader displacement. Older devices must fallback to a low-poly Icosahedron mesh to maintain 60fps.
- **Android Targets:** Devices with Adreno 640 or Mali-G77 GPUs (e.g., Samsung S10/Pixel 4 class).
- **Thermal Throttling:** Continuous execution of the WebGL loop alongside the Neural Engine (for on-device transcription in the Listener module) generates significant heat. The architecture must implement an "Idle State" logic (see Section 6.1) to mitigate thermal throttling.[10]

---

3. Functional Requirements: The Soul Sphere Visualization

The central visual component of the Experience module is the "Soul Sphere." This is not a static 3D model imported from Blender; it is a procedurally generated mesh that morphs in real-time. The visual language of the sphere is mathematically derived from the VAC vectors provided by the Versor engine.

3.1 Geometry Strategy and Vertex Manipulation

To represent the "Texture of Emotion," the system must manipulate the physical geometry of

the sphere. The user's "Arousal" state dictates the chaotic energy of the surface.

3.1.1 Base Geometry Selection

Standard SphereGeometry creates a grid of quads which distort unattractively at the poles (UV pinching).

- **Requirement:** The system shall use an IcosahedronGeometry with a subdivision detail level of 20+.
- **Justification:** Icosahedrons provide a uniform triangulation across the surface, ensuring that noise displacement is isotropic (equally distributed in all directions), preventing visual artifacts during rotation.[11]

3.1.2 The Math of Arousal (Vertex Shader)

Arousal (y-axis) represents the energy potential of the emotion.

- **Visual Metaphor:** Low arousal results in a smooth, polished surface (e.g., Calm, Boredom). High arousal results in a spiky, vibrating, chaotic surface (e.g., Excitement, Anger, Panic).[1]
- **Implementation:** A custom Vertex Shader is required. The system cannot rely on standard displacement maps because the noise must evolve temporally (animate) and spatially (change frequency) in real-time.
- **Algorithm:** The shader shall utilize 3D Simplex Noise or Perlin Noise.
    - Formula:

      $$P_{new} = P_{pos} + N \times (Noise(P_{pos} \times Freq + Time) \times Amp \times ArousalFactor)$$
    - Where P_{pos} is the vertex position, N is the normal vector, and Amp is the maximum displacement distance.
    - The ArousalFactor uniform is dynamically updated by the Versor engine.

3.1.3 Dynamic Normal Recalculation

A critical technical challenge in vertex displacement is lighting. When a shader moves vertices, the original surface normals (which determine how light bounces off the object) become incorrect, resulting in a smooth-looking sphere that has a spiky silhouette—a visual dissonance known as "flat shading on displaced geometry."

- **Requirement:** The shader must recalculate normals on-the-fly to reflect the new, jagged topography.
- **Technical Solution:** The implementation must utilize orthogonal derivatives in the fragment shader or neighbor-difference calculation in the vertex shader.
    - **Method A (Fragment Derivatives):** Using dFdx(vViewPosition) and dFdy(vViewPosition) allows for the calculation of the face normal of the displaced geometry. This requires the OES_standard_derivatives WebGL extension.[12]
    - Method B (Finite Difference): For higher fidelity, the vertex shader calculates the noise value for the current vertex and its immediate neighbors (tangent and bitangent), constructing a new normal vector via the cross product:

$$N_{new} = Normalize(Cross(Tangent, Bitangent))$$

3.2 Chromatic and Material Systems

The surface material is the primary vector for communicating Valence (x-axis) and Connection (z-axis).

3.2.1 Valence Mapping (Chromatic Interpolation)

Valence determines the color palette. The system must avoid abrupt color changes; emotions blend into one another.

- **Input:** Normalized Scalar $V \in [-1.0, 1.0]$.
- **Palette Definition:**
  - **Negative ($V \rightarrow -1.0$):** Deep Crimson (#8B0000), Charcoal (#36454F), Muted Indigo.
  - **Neutral ($V \approx 0.0$):** Slate Grey, Off-White.
  - **Positive ($V \rightarrow +1.0$):** Cyan (#00FFFF), Teal, Luminous Green.
- **Shader Logic:** The fragment shader uses the mix() function (linear interpolation) to blend between these defined vector colors based on the uValence uniform.[13]
  OpenGL Shading Language
  // GLSL Snippet for Valence Mixing
  vec3 color = mix(uNegativeColor, uPositiveColor, smoothstep(-1.0, 1.0, uValence));

3.2.2 The "Connection" Problem: Fresnel vs. Transmission

The z-axis (Connection) poses a significant rendering challenge. The conceptual requirement is for "disconnected" states to appear opaque and solid (like stone) and "connected" states to appear translucent and radiant (like spirit or glass).

- **Technical Constraint:** True transmission (glass refraction) using MeshPhysicalMaterial in Three.js is computationally prohibitive on mobile devices. It requires a dual-pass render (rendering the background to a buffer, then rendering the glass object) which often halves the frame rate.[15]
- **Performance Solution:** The Experience module shall implement a **"Fake Glow" Fresnel Shader**.
  - **Mechanism:** The Fresnel effect calculates the angle between the camera view vector and the surface normal. Surfaces facing the camera are transparent; surfaces varying away (the rim) become opaque/glowing.
  - **Logic:**
    - **High Connection:** Low Alpha + High Fresnel Power (Simulating a force field or bubble).
    - **Low Connection:** High Alpha + Low Fresnel Power (Simulating a solid diffuse object).
  - **Implementation:** This bypasses the need for the expensive transmission property, utilizing simple alpha blending (transparent: true) and additive color

math in the fragment shader to simulate inner light.[17]

3.3 The Versor Integration: Dynamic Orientation

The sphere must rotate to specific orientations representing the emotional state. This is distinct from standard Euler angle rotation (Pitch/Yaw/Roll), which suffers from Gimbal Lock.

- **Input Data:** The module receives a Unit Quaternion ($q_{target} = [w, x, y, z]$) from the Versor.
- **Interpolation Algorithm: Spherical Linear Interpolation (SLERP)**.
    - Unlike linear interpolation (LERP), which cuts through the chord of the sphere, SLERP follows the curvature of the 4D hypersphere. This ensures constant angular velocity, meaning the rotation doesn't speed up or slow down artificially during the turn.[1]
- **Animation Loop:** The React Three Fiber useFrame hook must handle this interpolation.
    - **Constraint:** Do not rely on React's useState or useEffect for the animation steps, as this triggers React reconciliation (re-renders) 60 times a second, killing performance. The interpolation must happen mutably on the Three.js object ref within the useFrame callback.[19]

---

4. Functional Requirements: The Haptic Feedback Engine

The Experience module is responsible for physically simulating the "Friction of Change." The document defines "Emotional Work" as the angular distance ϕ between two states. The device must vibrate to simulate the effort of this rotation.

4.1 Psychophysics of Emotional Work

The system quantifies the "size" of an emotional shift using the formula:

$$\phi = 2 \arccos(|w_{transition}|)$$

This angular distance drives the haptic engine.

- **Micro-Adjustments ($\phi < 15^\circ$):** The user is ruminating or stabilizing. Feedback should be imperceptible or extremely subtle.
- **The Pivot ($\phi > 45^\circ$):** A distinct shift in perspective. Feedback should be crisp and noticeable.
- **Radical Shifts ($\phi > 90^\circ$):** A complete change in orientation (e.g., from Shame to Self-Compassion). Feedback must be heavy, sustained, and convey "mass".[1]

4.2 Haptic Implementation Strategy

Standard vibration APIs are insufficient for this nuance. The implementation distinguishes

between iOS and Android capabilities.

4.2.1 High-Velocity Turn (The "Thud")

Triggered when the angular velocity of the rotation is high.

- **Library Selection:** react-native-haptics is strictly required over expo-haptics. Benchmarks indicate react-native-haptics has a latency of ~0.36ms compared to Expo's ~0.77ms for heavy impacts. This millisecond difference is perceptible when syncing haptics to visual apogees.[8]
- **Action:** Fire Haptics.impactAsync('heavy') at the precise moment the visual interpolation reaches 50% completion (the midpoint of the SLERP arc).

4.2.2 The "Heartbeat" (Stability Pattern)

When the user maintains a positive Connection state ($z > 0.5$) with stability (low angular velocity), the phone mimics a resting heartbeat.

- **Pattern:** Lub-Dub... Pause.
- **Timing:** 20ms ON, 100ms OFF, 40ms ON, 800ms OFF.
- **Android Implementation:** Android's Vibration.vibrate(pattern) API supports this natively with an array: ``.[20]
- **iOS Implementation:** iOS ignores custom duration arrays in the standard API. The Experience module must implement a custom recursive timer loop using Haptics.impactAsync('light') to simulate the pattern, or utilize a custom Native Module bridging CoreHaptics for waveform control.[21]

4.2.3 Flooding (Chaos Pattern)

Defined in the requirements as "Places We Go When Things Are Uncertain" [1], specifically "Overwhelm."

- **Trigger:** High Arousal ($y > 0.8$) + High Elasticity (Rapid state changes).
- **Sensation:** A chaotic, high-frequency buzzing.
- **Implementation:** A randomized vibration loop that varies intensity interval-by-interval, simulating a system warning or "flooding" engine.

---

5. Detailed Software Specifications (Blueprint)

This section provides the implementation-ready blueprint for the Claude coding agent, detailing file structures, shader code, and state logic.

5.1 Project Structure (Feature-First)

The directory structure should prioritize the modularity of the L.O.V.E. stack.

src/
features/
experience/

components/
SoulSphere/
index.tsx # R3F Mesh & Frame Loop
SoulMaterial.tsx # ShaderMaterial Definition
shaders/
vertex.glsl # Arousal Displacement Logic
fragment.glsl # Valence/Connection Logic
HapticManager.tsx # Headless component for vibration logic
hooks/
useEmotionalPhysics.ts # Math: SLERP & Angular Velocity
useTextureLoader.ts # KTX2 Asset Management
store/
useExperienceStore.ts # Zustand State (VAC vectors)

5.2 GLSL Shader Specifications

These shaders are the core IP of the Experience module.

5.2.1 Vertex Shader (vertex.glsl)

This shader handles the vertex displacement (Arousal).

OpenGL Shading Language

```
precision mediump float;

// Uniforms driven by React State
uniform float uTime;
uniform float uArousal; // 0.0 to 1.0

// Standard Three.js attributes
varying vec2 vUv;
varying vec3 vNormal;
varying float vDisplacement; // Passed to fragment for peak coloring

// Simplex Noise Function (Imported via three-stdlib chunks)
#pragma glslify: snoise3 = require(glsl-noise/simplex/3d)

void main() {
  vUv = uv;
  vec3 pos = position;

  // Arousal determines Noise Frequency and Amplitude
  float noiseFreq = 1.5 + (uArousal * 2.0);
```

```
  float noiseAmp = 0.2 * uArousal; // Zero arousal = perfect sphere

  // Animate noise over time
  vec3 noisePos = vec3(pos.x * noiseFreq + uTime, pos.y * noiseFreq, pos.z * noiseFreq);
  float noiseValue = snoise3(noisePos);

  // Displace along the normal vector
  float displacement = noiseValue * noiseAmp;
  pos += normal * displacement;

  vDisplacement = displacement;

  // Note: Normals are NOT recalculated here for performance.
  // We rely on high-poly density or fragment derivatives.
  vNormal = normal;

  gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
}
```

5.2.2 Fragment Shader (fragment.glsl)

This shader handles the "Fake Glow" (Connection) and Color (Valence).

OpenGL Shading Language

```
precision mediump float;

uniform float uValence;   // -1.0 to 1.0
uniform float uConnection; // -1.0 to 1.0
uniform vec3 uColorNeg;   // #8B0000
uniform vec3 uColorPos;   // #00FFFF
uniform vec3 uViewVector; // Camera Position - Vertex Position

varying vec3 vNormal;
varying float vDisplacement;

void main() {
  // 1. Valence Mapping (Color)
  // Remap [-1, 1] to
  float mixFactor = (uValence + 1.0) * 0.5;
  vec3 baseColor = mix(uColorNeg, uColorPos, mixFactor);
```

```
  // Add highlight to displaced peaks (Arousal Visualization)
  baseColor += vec3(vDisplacement * 2.0);

  // 2. Connection Mapping (Fresnel Glow)
  // Calculate Rim Lighting intensity
  vec3 normal = normalize(vNormal);
  vec3 viewDir = normalize(uViewVector);
  float fresnelDot = dot(viewDir, normal);

  // Invert dot: 1.0 at edges, 0.0 at center
  float fresnelTerm = clamp(1.0 - fresnelDot, 0.0, 1.0);

  // Connection Intensity scales the glow
  // High Connection (1.0) = Strong Glow
  float glowIntensity = pow(fresnelTerm, 2.0) * (uConnection + 1.2);

  vec3 finalColor = baseColor + (vec3(0.8, 0.9, 1.0) * glowIntensity);

  // 3. Opacity Logic
  // High Connection = Translucent
  float alpha = 1.0;
  if (uConnection > 0.0) {
     alpha = 0.9 - (uConnection * 0.5) + fresnelTerm;
  }

  gl_FragColor = vec4(finalColor, alpha);
}
```

5.3 State Management and Performance Logic

To maintain 60fps, the Javascript thread must not be blocked by React reconciliation.

- **Zustand Store:** The useExperienceStore shall hold the *target* vectors. It does NOT hold the interpolated frame-by-frame values.
- **Transient Updates:** The SoulSphere component shall subscribe to the store selectively.
  TypeScript
  ```
  // useExperienceStore.ts
  import { create } from 'zustand'

  interface State {
    targetVAC: [number, number, number]
    targetQuaternion: [number, number, number, number]
    setTarget: (vac: [number, number, number], q: [number, number, number, number]) =>
  void
  ```

```
}
```

- **Frame Loop:**
```
TypeScript
useFrame((state, delta) => {
  // Interpolate CURRENT mesh values toward TARGET store values
  // This runs on the JS thread but bypasses React Render
  meshRef.current.material.uniforms.uValence.value = THREE.MathUtils.lerp(
    meshRef.current.material.uniforms.uValence.value,
    targetVAC,
    delta * 2.0
  );
  // SLERP Rotation
  meshRef.current.quaternion.slerp(targetQuaternion, delta * 1.5);
})
```

---

6. Performance Optimization and Asset Strategy

The Experience module must run efficiently on constrained mobile hardware.

6.1 On-Demand Rendering

A continuous 60fps loop drains battery rapidly. The user may spend minutes reading a journal entry where the emotional state (and thus the sphere) is static.

- **Requirement:** Implement frameloop="demand" on the R3F <Canvas>.
- **Logic:** The canvas only renders when visual props change.
- **Invalidation Strategy:**
  - When the Listener detects voice input: invalidate() (Start Rendering).
  - When the Versor sends a new target quaternion: invalidate().
  - During the SLERP transition: The useFrame loop keeps the invalidation active until the quaternion delta approaches zero (≈ 0.001).
  - Once the target is reached, the loop stops, saving battery.[10]

6.2 Asset Compression (KTX2)

If the Experience module includes environmental textures (e.g., background "Skyboxes" corresponding to the 13 Emotion Categories), standard PNG/JPEG textures will cause frame drops during loading due to texture decoding on the main thread.

- **Requirement:** All textures must be converted to **KTX2 (Basis Universal)** format.
- **Benefit:** KTX2 is transcoded directly on the GPU, bypassing CPU decoding. It reduces VRAM usage by up to 70%, preventing crashes on iOS Safari/WebViews which have strict memory limits.[22]

- **Implementation:** Use the useKTX2 hook from @react-three/drei and ensure the gl context is passed to the KTX2Loader.

6.3 Instancing Strategy

If the visualization expands to show a "History of Emotions" (multiple spheres representing past states), the system must not create unique draw calls for each sphere.

- **Requirement:** Use InstancedMesh for rendering historical data points.
- **Optimization:** A single draw call can render 1000+ past emotional states, with each instance having a unique color and position matrix, but sharing the same geometry and material shader.[10]

---

7. Risk Management and Mitigation

7.1 Risk: "New Architecture" Instability

- **Description:** As identified in Section 2.1, the Fabric renderer is currently incompatible with expo-gl.
- **Mitigation:** The project README must explicitly document the requirement for RCT_NEW_ARCH_ENABLED=0. Continuous Integration (CI) pipelines must verify this flag before building release binaries.

7.2 Risk: Haptic Desensitization

- **Description:** Overuse of the "Heartbeat" or "Flooding" patterns may annoy the user or drain the battery.
- **Mitigation:**
  - **Decay Function:** Haptics should fade out after 5 seconds of state stability.
  - **User Override:** A global "Quiet Mode" toggle in the settings must strictly disable all calls to the Haptic Manager.

7.3 Risk: Color Blindness Accessibility

- **Description:** The red-green spectrum (Crimson to Teal) used for Valence is problematic for Deuteranopia.
- **Mitigation:** The fragment shader shall include a "Colorblind Mode" uniform (uAccessibilityMode). When enabled, the palette shifts to a Blue-Orange spectrum (High Contrast), ensuring Valence is distinguishable by all users.

---

8. Conclusion

The Experience Module serves as the bridge between the rigorous algebra of Project L.O.V.E. and the felt reality of the user. By adhering to this specification—specifically the use of legacy-compatible React Native configurations, shader-based geometry manipulation, and

pyschophysically tuned haptics—the system will deliver on its promise: to not just track mood, but to visualize the trajectory of the human soul with mathematical precision and artistic empathy.

Works cited

1. L.O.V.E. Project Software Requirements.pdf
2. Issue Installing react-three/fiber@alpha react-three/drei dependency - Stack Overflow, accessed December 2, 2025, https://stackoverflow.com/questions/79373690/issue-installing-react-three-fiberalpha-react-three-drei-dependency
3. About the New Architecture - React Native, accessed December 2, 2025, https://reactnative.dev/architecture/landing-page
4. Expo-gl issues with sdk52 when trying to use @react-three/fiber/native - Reddit, accessed December 2, 2025, https://www.reddit.com/r/expo/comments/1jhhb8v/expogl_issues_with_sdk52_when_trying_to_use/
5. Is anyone else having issues with the new architecture? : r/reactnative - Reddit, accessed December 2, 2025, https://www.reddit.com/r/reactnative/comments/1hvxick/is_anyone_else_having_issues_with_the_new/
6. Reference - Expo Documentation, accessed December 2, 2025, https://docs.expo.dev/versions/latest/
7. React19 compatibility · Issue #2260 · pmndrs/drei - GitHub, accessed December 2, 2025, https://github.com/pmndrs/drei/issues/2260
8. A high-performance React Native library for iOS haptics and Android vibration effects - GitHub, accessed December 2, 2025, https://github.com/mhpdev-com/react-native-haptics
9. React Native Haptics: A Fast, High-Performance Library for iOS Haptics and Android Vibration Effects | by M.H.Pousti | Medium, accessed December 2, 2025, https://medium.com/@hpousty/react-native-haptics-a-fast-high-performance-library-for-ios-haptics-and-android-vibration-ec107f97a7ee
10. Scaling performance - React Three Fiber, accessed December 2, 2025, https://r3f.docs.pmnd.rs/advanced/scaling-performance
11. The Study of Shaders with React Three Fiber - Maxime Heckel Blog, accessed December 2, 2025, https://blog.maximeheckel.com/posts/the-study-of-shaders-with-react-three-fiber/
12. Calculating vertex normals after displacement in the vertex shader - three.js forum, accessed December 2, 2025, https://discourse.threejs.org/t/calculating-vertex-normals-after-displacement-in-the-vertex-shader/16989
13. [GLSL] Smoothstep - Medium, accessed December 2, 2025, https://medium.com/@vnflqkq/glsl-smoothstep-6554e62be517
14. THREE.JS SHADER: Apply color gradient blend to geometry on multiple axes?,

accessed December 2, 2025, https://stackoverflow.com/questions/73397117/three-js-shader-apply-color-gradient-blend-to-geometry-on-multiple-axes
15. Performance Difference in android device - Questions - three.js forum, accessed December 2, 2025, https://discourse.threejs.org/t/performance-difference-in-android-device/21823
16. MeshTransmissionMaterial poor performances URGENT - Questions - three.js forum, accessed December 2, 2025, https://discourse.threejs.org/t/meshtransmissionmaterial-poor-performances-urgent/68566
17. A fake glow material for react three fiber. - GitHub, accessed December 2, 2025, https://github.com/ektogamat/fake-glow-material-r3f
18. three.js glow shader - Kade Keith, accessed December 2, 2025, https://kadekeith.me/2017/09/12/three-glow.html
19. Basic Animations - React Three Fiber, accessed December 2, 2025, https://r3f.docs.pmnd.rs/tutorials/basic-animations
20. Proper different amplitude values for vibration - Stack Overflow, accessed December 2, 2025, https://stackoverflow.com/questions/65257664/proper-different-amplitude-values-for-vibration
21. Vibration - React Native, accessed December 2, 2025, https://reactnative.dev/docs/vibration
22. Fix Loading Model Freezes with Three.js & React - YouTube, accessed December 2, 2025, https://www.youtube.com/watch?v=Wt3iEenj_Xw
23. Fix Loading Model Freezes with Three.js & React - Wawa Sensei, accessed December 2, 2025, https://wawasensei.dev/tuto/fix-loading-model-freezes-threejs-react-ktx2
24. Minimal instancedMesh example · pmndrs react-three-fiber · Discussion #761 - GitHub, accessed December 2, 2025, https://github.com/pmndrs/react-three-fiber/discussions/761